



Facultad de Inteligencia Artificial e Ingenierías

Análisis y Diseño de Algoritmos — 2026-1

Manual Técnico

Proyecto K-QGMIP

Extensión de KGeoMIP y KQNodes a k-Particiones

Juan Sebastián García Hincapié · ID: SebastianG210

Institución: Universidad de Caldas

Materia: Análisis y Diseño de Algoritmos

Versión: 1.0

Fecha: Junio 2026

Índice

1. Resumen Ejecutivo

1.1. Descripción del Problema

El presente proyecto aborda la extensión del problema de la **Partición de Mínima Información** (MIP, *Minimum Information Partition*) al caso general de k -particiones dentro del marco de la **Teoría de la Información Integrada** (IIT). El problema consiste en determinar cómo dividir un sistema de n variables binarias en k partes independientes de manera que se minimice la pérdida de información integrada. Formalmente, se busca la partición del sistema V en k subconjuntos disjuntos $S_1, S_2, \dots, S_k$ que minimice la discrepancia entre la dinámica del sistema completo y la dinámica reconstruida desde las partes.

1.2. Enfoque Algorítmico

Se implementaron dos estrategias que extienden los algoritmos existentes de bi-particiones:

- **KGeoMIP**: Extensión del algoritmo geométrico GeoMIP. Aprovecha la representación del espacio de estados como hipercubo n -dimensional y una tabla de costos de transiciones para generar y evaluar particiones candidatas en k grupos ($k \in \{2, 3, 4, 5\}$).
- **KQNodes**: Extensión del algoritmo QNodes basado en el método de Queyranne para optimización submodular. Utiliza un oráculo Zeta precomputado y búsqueda exhaustiva o heurística greedy sobre las $S(n, k)$ posibles k -particiones.

Ambas estrategias adaptan automáticamente su modo de búsqueda (exacto vs. heurístico) según el número de Stirling de segundo tipo $S(n, k)$ respecto a un presupuesto configurable (por defecto 10,000 evaluaciones).

1.3. Principales Resultados

Se ejecutaron benchmarks sobre sistemas de 5 a 22 variables (redes N5B, N10A, N10B, N15A, N20A, N21A, N22A). Los experimentos validan:

- Para $k = 2$, ambas extensiones reproducen exactamente los resultados de GeoMIP y QNodes originales (validaciones TV-02 y TV-03).
- KQNodes garantiza optimalidad exacta cuando $S(n, k) \leq 10,000$.
- KGeoMIP ofrece tiempos menores con búsqueda heurística guiada geoméricamente.

1.4. Limitaciones y Recomendaciones

La complejidad combinatoria de $S(n, k)$ limita la búsqueda exacta a sistemas de tamaño moderado. Para $n \geq 15$ y $k \geq 3$, ambos algoritmos operan en modo heurístico. Se recomienda KQNodes cuando se requiere exactitud en sistemas pequeños, y KGeoMIP cuando prima la velocidad en sistemas grandes.

2. Fundamentos Teóricos

2.1. Teoría de la Información Integrada

La Teoría de la Información Integrada [?] proporciona un marco matemático para cuantificar la conciencia en sistemas físicos mediante Φ . Un componente fundamental es el concepto de MIP (*Minimum Information Partition*): la división del sistema que minimiza la pérdida de información al reconstruir la dinámica desde partes independientes.

2.2. Definición Formal de k-Partición

Sea $V = \{v_1, v_2, \dots, v_n\}$ un sistema de n variables binarias con Matriz de Probabilidad de Transición (TPM) $P(V_{t+1} | V_t)$.

Definición 2.1 (k-Partición). Una **k-partición** de V es una familia de subconjuntos no vacíos $\Pi = \{S_1, S_2, \dots, S_k\}$ tal que:

$$S_i \cap S_j = \emptyset \quad \forall i \neq j \quad (1)$$

$$S_1 \cup S_2 \cup \dots \cup S_k = V \quad (2)$$

$$S_i \neq \emptyset \quad \forall i \in \{1, \dots, k\} \quad (3)$$

Ejemplo ($n = 4, k = 3$): Si $V = \{A, B, C, D\}$, la partición $\Pi = \{\{A, B\}, \{C\}, \{D\}\}$ satisface las condiciones (??)–(??).

2.3. Función de Pérdida y Earth Mover's Distance

La pérdida de una k -partición $\Pi = \{S_1, \dots, S_k\}$ se cuantifica con la **Earth Mover's Distance** (EMD) entre la distribución original del sistema y la reconstruida desde las partes independientes:

$$p_{\Pi}(V_{t+1}) = \bigotimes_{i=1}^k p(S_i^{t+1}) \quad (4)$$

La función de pérdida objetivo es:

$$\delta_k(\Pi) = \text{EMD}\left(p(V_{t+1}), \bigotimes_{i=1}^k p(S_i^{t+1})\right) = \sum_{j=1}^n \left| p_j^{\text{original}} - \bar{p}_j^{\Pi} \right| \quad (5)$$

donde $\bar{p}_j^{\Pi}$ es el promedio ponderado de las distribuciones marginales por grupo. El **k-MIP** es:

$$\Pi^* = \arg\min_{\Pi \in \mathcal{P}(V, k)} \delta_k(\Pi) \quad (6)$$

2.4. Números de Stirling de Segundo Tipo

El número de k -particiones de n elementos está dado por:

$$S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n \quad (7)$$

Cuadro 1: Valores de $S(n, k)$ para diferentes n y k .

n	$k = 2$	$k = 3$	$k = 4$	$k = 5$
5	15	25	10	1
10	511	9,330	34,105	55,980
15	16,383	2,375,101	$> 10^7$	$> 10^8$
20	524,287	$> 10^8$	$> 10^{11}$	$> 10^{14}$

2.5. Representación Geométrica: Hipercubo n -dimensional

El espacio de estados $\{0, 1\}^n$ se corresponde con los 2^n vértices de un hipercubo n -dimensional. La **distancia de Hamming** es:

$$d_H(s, s') = \sum_{i=1}^n |s_i - s'_i| \quad (8)$$

El costo de transición en GeoMIP se define como:

$$c_i(s, s') = \frac{1}{2^{d_H(s, s')}} \left(|X_i[s] - X_i[s']| + \sum_{v \in \partial^*(s', s)} c_i(s, v) \right) \quad (9)$$

2.6. Extensión a k -Particiones

La extensión de GeoMIP y QNodes se fundamenta en tres observaciones:

- 1. Reutilización de la tabla de costos:** Se calcula una sola vez en $\mathcal{O}(n \cdot 2^n)$ y es válida para cualquier k .
- 2. Generalización de la pérdida:** Para $k > 2$, se usa el promedio de distribuciones marginales (ecuación (??)).
- 3. Consistencia para $k = 2$:** Las extensiones reproducen exactamente los resultados originales de bi-partición.

3. Arquitectura del Software

3.1. Visión General

El sistema sigue una arquitectura modular basada en el **Patrón Strategy** [?]. La clase abstracta **SIA** define la interfaz común y cada estrategia concreta hereda e implementa su propio algoritmo de búsqueda.

3.2. Diagrama de Paquetes

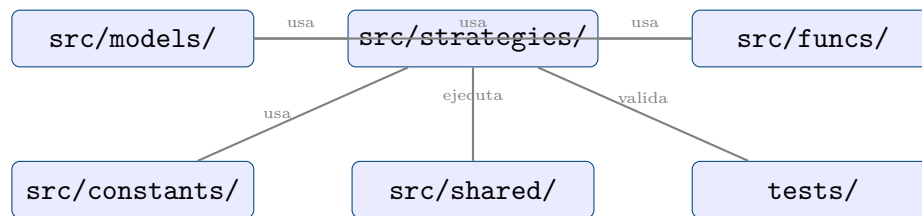


Figura 1: Diagrama de paquetes del proyecto K-QGMIP.

3.3. Diagrama de Clases

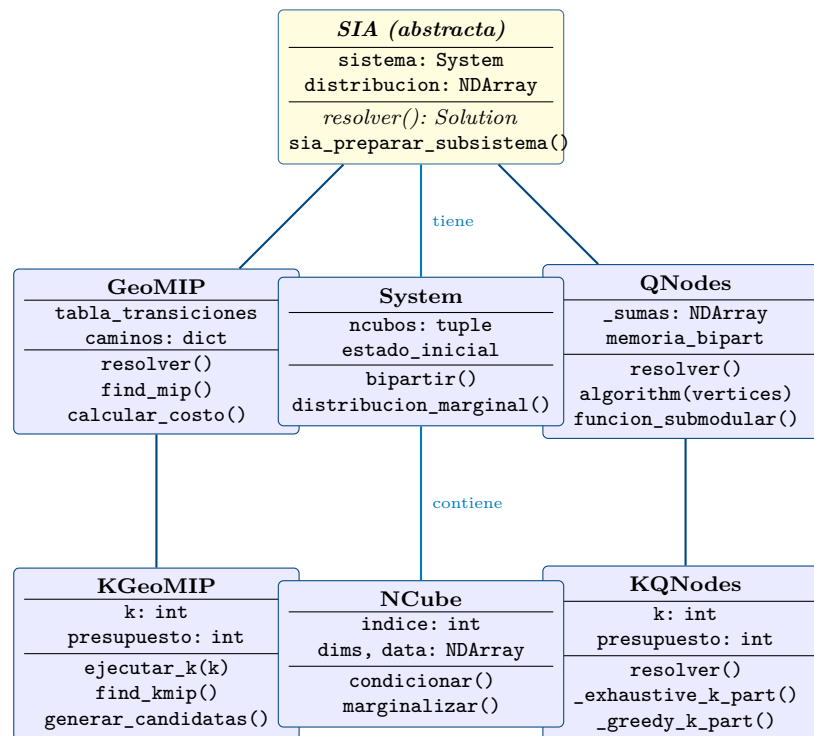


Figura 2: Diagrama de clases UML. Triángulo abierto = herencia; rombo = composición.

3.4. Estructura de Directorios del Código

Listing 1: Árbol de directorios del proyecto.

```

1 proyectoAlgoritmos/
2     src/
3         models/
4             base/
5                 sia.py           # Clase abstracta SIA
6                 application.py  # Singleton global
7             core/
8                 system.py       # Clase System
9                 ncube.py        # Clase NCube
10                solution.py     # Clase Solution
11            strategies/
12                GeoMIP/
13                    geomip.py    # GeoMIP original
14                    kgeomip.py  # KGeoMIP (k-
particiones)
15                QNodes/
16                    qnodes.py    # QNodes original
17                    kqnodes.py  # KQNodes (k-particiones)
18            funcs/
19                iit.py           # emd_efecto, literales
20                format.py       # Formateo de particiones
21                analytic.py     # Transformada Zeta
22            constants/base.py   # ACTUAL, EFFECT, ...
23            shared/
24                input/          # Archivos CSV de casos
25                output/         # Resultados por red
26                run/
27                    benchmark.py # Orquestador principal
28                    dashboard.py # Visualizaci n HTML
29        tests/
30            test_qnodes_vs_phi.py
31        pyproject.toml

```

3.5. Patrones de Diseño Aplicados

Strategy SIA define la interfaz `resolver(): Solution`. Cada estrategia la implementa de manera independiente.

Template Method

`sia_preparar_subsistema()` en la clase base define el flujo de preparación; las subclases lo invocan y aplican su propia lógica de búsqueda.

Singleton

`Application` mantiene la configuración global (semilla NumPy, tipo de EMD, notación) como única instancia.

Memoización

`NCube.memo` cachea marginalizaciones. `memoria_kparticiones` y `memoria_bipart` evitan recomputar evaluaciones repetidas.

3.6. Decisiones Arquitectónicas Clave

1. **Reutilización de la tabla de costos:** Se construye una sola vez en $\mathcal{O}(n \cdot 2^n)$ y se reutiliza para todos los valores de k .
2. **Oráculo Zeta:** Precomputa las sumas de hiper-cara una vez, reduciendo cada consulta del oráculo a $\mathcal{O}(1)$ amortizado.
3. **Umbral exacto/heurístico:** $S(n, k) = 10,000$ equilibra exactitud y tiempo de ejecución (< 1 s para casos típicos).
4. **Dos interfaces:** `resolver()` para flujos optimizados y `aplicar_estrategia()` para compatibilidad legada con TPM cruda.

4. Diseño Algorítmico

4.1. Visión General

KGeoMIP y KQNodes comparten la misma estructura de alto nivel: (1) preparar estructuras de datos, (2) generar particiones candidatas, (3) evaluar cada candidata con EMD, y (4) retornar la de menor pérdida. La diferencia fundamental radica en cómo se generan las candidatas y qué oráculo se usa para evaluarlas.

4.2. Algoritmo GeoMIP (Base)

GeoMIP construye la tabla de costos navegando el hipercubo por niveles de distancia Hamming.

Algorithm 1: GeoMIP: búsqueda geométrico-topológica del bi-MIP.

Entrada: NCubos, estado inicial s_0 , estado final $s_f = 1 - s_0$

Salida: MIP: bi-partición de mínima pérdida

```

1 caminos[0]  $\leftarrow \{s_0\}$ 
2 tabla[( $s_0, s_0$ )]  $\leftarrow \mathbf{0}$ 
3 para nivel  $\leftarrow 1$  hasta  $n$  hacer
4     para  $s \in \text{caminos}[\text{nivel} - 1]$  hacer
5         para  $i \leftarrow 1$  hasta  $n$  hacer
6             si  $s[i] \neq s_f[i]$  entonces
7                  $s' \leftarrow s$  con  $s'[i] \leftarrow s_f[i]$ 
8                 si  $s'$  no visitado entonces
9                     calcular  $c(s_0, s')$  según ec. (4)
10                    caminos[nivel] +=  $\{s'\}$ 
11 candidatos  $\leftarrow \text{identificar\_particiones\_optimas}()$ 
12 para ( $\text{pres}, \text{fut}$ )  $\in \text{candidatos}$  hacer
13     dist  $\leftarrow \text{bipartir}(\text{fut}, \text{pres}).\text{distribucion\_marginal}()$ 
14     emd  $\leftarrow \text{EMD}(\text{dist}, p_{\text{orig}})$ 
15 retornar  $\arg \min_{\Pi} \text{emd}[\Pi]$ 

```

4.3. Algoritmo KGeoMIP

KGeoMIP reutiliza la tabla de costos de GeoMIP y añade generación y evaluación de k -particiones candidatas.

Algorithm 2: KGeoMIP: búsqueda de k -MIP geométrico.**Entrada:** Sistema V , número de particiones k , presupuesto $B = 10,000$ **Salida:** k -MIP

```

1  __preparar_geometria()           // tabla de costos Hamming,  $O(n \cdot 2^n)$ 
2   $v \leftarrow \{(\text{EFECTO}, i)\} \cup \{(\text{ACTUAL}, d)\}$ 
3   $S(|v|, k) \leftarrow$  calcular según ec. (5)
4  si  $S(|v|, k) \leq B$  entonces
5  |    $C \leftarrow$  __candidatas_exactas( $k$ )           // combin. + Hamming + exhaustivo
6  sino
7  |    $C \leftarrow$  __candidatas_heuristicas( $k$ )           // solo geométricas
8   $C \leftarrow$  deduplica( $C$ )
9  para  $\Pi \in C$  hacer
10 |   __evaluar_k_particion( $\Pi$ )
11 retornar  $\arg \min_{\Pi} \text{memoria}[\Pi].emd$ 

```

4.3.1. Evaluación de una k -Partición**Algorithm 3:** __evaluar_k_particion: cálculo de EMD para una k -partición.**Entrada:** Partición $\Pi = \{G_1, \dots, G_k\}$ **Salida:** EMD registrado en memoria

```

1  si  $k = 2$  entonces
2  |    $\text{fut} \leftarrow \{i : (\text{EFECTO}, i) \in G_1 \cup G_2\}$ 
3  |    $\text{pres} \leftarrow \{d : (\text{ACTUAL}, d) \in G_1 \cup G_2\}$ 
4  |    $\text{dist} \leftarrow \text{bipartir}(\text{fut}, \text{pres}).\text{distribucion\_marginal}()$ 
5  |    $\text{emd} \leftarrow \text{EMD}(\text{dist}, p_{\text{orig}})$ 
6  sino
7  |   para  $G_i \in \Pi$  hacer
8  |   |    $f_i \leftarrow \{i : (\text{EFECTO}, i) \in G_i\}; \quad m_i \leftarrow \{d : (\text{ACTUAL}, d) \in G_i\}$ 
9  |   |    $\text{dists}[i] \leftarrow \text{bipartir}(f_i, m_i).\text{distribucion\_marginal}()$ 
10 |    $\bar{p} \leftarrow \frac{1}{k} \sum_i \text{dists}[i]$ 
11 |    $\text{emd} \leftarrow \text{EMD}(\bar{p}, p_{\text{orig}})$ 
12 registrar  $(\Pi, \text{emd})$  en memoria

```

4.4. Algoritmo QNodes (Base)

QNodes precomputa la transformada Zeta sobre $\delta = H - p$ y ejecuta el MAO (*Maximum Adjacency Ordering*).

Algorithm 4: QNodes: MAO con oráculo Zeta.**Entrada:** Sistema preparado**Salida:** MIP

```

1 __preparar_oraculo()                // transf. Zeta:  $O(D \cdot N \cdot 2^D)$ 
2  $vert \leftarrow \{(\text{ACTUAL}, d)\} \cup \{(\text{EFECTO}, i)\}$ 
3 para  $v \in vert$  hacer
4   | registrar_candidato( $v$ )           // pre-pass: singletons
5 mientras  $|vert| > 1$  hacer
6   |  $\omega \leftarrow [vert[0]]$ ;  $\Delta \leftarrow vert[1:]$ 
7   | mientras  $|\Delta| > 1$  hacer
8     |  $\delta^* \leftarrow \arg \min_{\delta \in \Delta} [\text{EMD}(\omega \cup \{\delta\}) - \text{EMD}(\{\delta\})]$ 
9     |  $\omega += \delta^*$ ;  $\Delta -= \delta^*$ 
10  | registrar_candidato( $\Delta[0]$ )       // nodo colgante
11  |  $vert \leftarrow \omega[: -1] \cup \{\omega[-1] \oplus \Delta[0]\}$  // contraer
12 retornar  $\arg \min_{\Pi} memoria[\Pi].emd$ 

```

4.5. Algoritmo KQNodes**Algorithm 5:** KQNodes: búsqueda de k -MIP con oráculo Zeta.**Entrada:** Sistema, k , presupuesto $B = 10,000$ **Salida:** k -MIP

```

1 __preparar_oraculo()                // transformada Zeta
2  $vert \leftarrow \text{presente} + \text{futuro}$ ;  $S(|vert|, k) \leftarrow \text{calcular}$ 
3 si  $S(|vert|, k) \leq B$  entonces
4   |  $\Pi^* \leftarrow \text{\_exhaustive\_k\_partition}(vert, k)$ 
5 sino
6   |  $\Pi^* \leftarrow \text{\_greedy\_k\_partition}(vert, k)$ 
7  $\delta^* \leftarrow \text{\_compute\_partition\_loss}(\Pi^*)$ 
8 retornar  $Solution(perdida=\delta^*, partición=\Pi^*)$ 

```

4.5.1. Búsqueda Exhaustiva

Algorithm 6: `_exhaustive_k_partition`: enumera las $S(n, k)$ particiones.

Entrada: Vértices, k

Salida: k -partición óptima

```

1  $\Pi^* \leftarrow \emptyset$ ;  $\delta^* \leftarrow +\infty$ 
2 para  $\Pi \in \text{generar\_particiones}(\text{vert}, k)$  hacer
3    $\delta \leftarrow \text{\_compute\_partition\_loss}(\Pi)$ 
4   si  $\delta < \delta^*$  entonces
5      $\delta^* \leftarrow \delta$ ;  $\Pi^* \leftarrow \Pi$ 
6 retornar  $\Pi^*$ 

```

4.5.2. Particionamiento Greedy (modo heurístico)

Algorithm 7: `_greedy_k_partition`: heurística greedy por iteraciones del MAO.

Entrada: Vértices V , k

Salida: k -partición aproximada

```

1 grupos  $\leftarrow []$ ;  $V_r \leftarrow V$ 
2 para  $i \leftarrow 1$  hasta  $k - 1$  hacer
3   mip  $\leftarrow \text{algorithm}(V_r)$  // MAO sobre el resto
4    $G \leftarrow \text{aplanar}(\text{mip})$ 
5   grupos  $\text{+= } G$ ;  $V_r \leftarrow V_r \setminus G$ 
6 grupos  $\text{+= } V_r$  // último grupo = todo el resto
7 retornar grupos

```

4.6. Estructuras de Datos

4.6.1. N-Cubo

Cada N-Cubo almacena la TPM de un nodo como hipercubo n -dimensional. Los accesos usan notación *little-endian*: el bit menos significativo corresponde al último eje.

Listing 2: Estructura esencial del N-Cubo.

```

1 @dataclass(frozen=True)
2 class NCube:
3     indice: int # indice del nodo
4     dims: NDArray[np.int8] # dimensiones activas
5     data: np.ndarray # hipercubo n-dimensional
6     memo: dict = field(default_factory=dict)
7
8     def marginalizar(self, ejes: NDArray) -> "NCube":
9         resultado = self.data
10        for eje in sorted(ejes, reverse=True):
11            resultado = resultado.mean(axis=eje)

```

```
12     return NCube(indice=self.indice, dims=..., data=resultado)
```

4.6.2. Representación de Vértices y Particiones

Un vértice es la 2-tupla (tiempo, índice) con tiempo $\in \{0 = \text{ACTUAL}, 1 = \text{EFECTO}\}$. Una k -partición es una lista de k grupos, donde cada grupo es una lista de vértices:

Listing 3: Ejemplo de 3-partición de 4 nodos.

```
1  particion = [  
2      [(1,0), (0,0)],           # Grupo 1: Futuro A + Presente A  
3      [(1,1), (0,1)],           # Grupo 2: Futuro B + Presente B  
4      [(1,2), (1,3), (0,2), (0,3)], # Grupo 3: resto  
5  ]
```

5. Análisis de Complejidad

5.1. Complejidad Temporal

Sea n el número de variables, $N = 2^n$ el número de estados, y D la dimensión del subsistema analizado ($D \leq n$). La Tabla ?? resume la complejidad temporal de cada algoritmo.

Cuadro 2: Complejidad temporal de los algoritmos implementados.

Algoritmo	Precómputo	Búsqueda	Observaciones
BruteForce	$\mathcal{O}(1)$	$\mathcal{O}(2^{m+n} \cdot n)$	Baseline; inviable $n > 15$
GeoMIP	$\mathcal{O}(D \cdot N)$	$\mathcal{O}(D^2)$	Tabla Hamming + candidatos
KGeoMIP (exacto)	$\mathcal{O}(D \cdot N)$	$\mathcal{O}(S(D, k) \cdot D^2)$	Enumera candidatos
KGeoMIP (heurístico)	$\mathcal{O}(D \cdot N)$	$\mathcal{O}(C \cdot D^2)$	$C \ll S(D, k)$
QNodes	$\mathcal{O}(D \cdot N)$	$\mathcal{O}(D^3 \cdot N)$	Oráculo Zeta + MAO
KQNodes (exacto)	$\mathcal{O}(D \cdot N)$	$\mathcal{O}(S(D, k) \cdot D)$	Enumera y evalúa
KQNodes (greedy)	$\mathcal{O}(D \cdot N)$	$\mathcal{O}(k \cdot D^3 \cdot N)$	$k - 1$ iteraciones MAO

5.1.1. Análisis del Precómputo

Tanto el oráculo Zeta (QNodes) como la tabla de costos (GeoMIP) tienen costo $\mathcal{O}(D \cdot N)$ en la práctica, dominado por el recorrido de los 2^D estados del hipercubo. Para $D = 20$, esto implica $\sim 2 \times 10^7$ operaciones flotantes, manejable en hardware moderno.

5.1.2. Análisis de la Búsqueda

Para KQNodes en modo exhaustivo, la complejidad es $\mathcal{O}(S(n, k))$ evaluaciones. El umbral $B = 10,000$ garantiza viabilidad computacional (ver Tabla ?? en la sección de Fundamentos). En modo greedy, la complejidad es $\mathcal{O}(k \cdot D^3 \cdot N)$, lineal en k .

5.2. Complejidad Espacial

- **Sistema y N-Cubos:** $\mathcal{O}(n \cdot 2^n)$ para las n matrices de transición.
- **Oráculo Zeta (KQNodes):** $\mathcal{O}(N \cdot 2^D)$ para la matriz de sumas de cara.
- **Tabla de costos (KGeoMIP):** $\mathcal{O}(2^{2n})$ en el peor caso (todos los pares de estados).
- **Memoria de particiones:** $\mathcal{O}(S(n, k))$ en modo exacto.

5.3. Análisis de Casos

Mejor caso:

Sistema con n pequeño (≤ 10) y $k = 2$. Ambas estrategias operan en tiempo cercano a $\mathcal{O}(n^3)$ (MAO base / tabla Hamming sin heurísticas adicionales).

Peor caso:

Sistema con n grande (≥ 20) y $k = 5$. KQNodes entra en modo greedy ($\mathcal{O}(k \cdot D^3 \cdot N)$); KGeoMIP genera muchas candidatas geométricas antes de podar.

5.4. Comparación con Alternativas

Cuadro 3: Comparación de complejidad y tiempo estimado para $n = 20$, $k = 3$.

Método	Exacto	Complejidad total	Tiempo est.
Búsqueda exhaustiva	Sí	$\mathcal{O}(2^{m+n} \cdot n)$	inviable
GeoMIP ($k = 2$)	Heur.	$\mathcal{O}(D \cdot N + D^2)$	$< 0,5$ s
QNodes ($k = 2$)	Heur.	$\mathcal{O}(D \cdot N + D^3 N)$	< 2 s
KGeoMIP ($k = 3$)	Heur.	$\mathcal{O}(D \cdot N + C \cdot D^2)$	~ 1 s
KQNodes ($k = 3$)	Heur.	$\mathcal{O}(D \cdot N + k \cdot D^3 N)$	~ 3 s

6. Detalles de Implementación

6.1. Métodos Principales de KGeoMIP

ejecutar_k(k: int) ->Solution

Punto de entrada principal. Para $k = 2$, delega directamente a GeoMIP (garantiza consistencia TV-02). Para $k > 2$, invoca `find_kmip()`.

find_kmip() ->list[list[Vertice]]

Orquesta la generación de candidatas y su evaluación. Retorna la k -partición con menor EMD registrado en `memoria_kparticiones`.

generar_k_particiones_candidatas(k)

Selecciona modo exacto u heurístico. Modo exacto: (i) separación por combinaciones de futuros, (ii) niveles Hamming, (iii) enumeración exhaustiva si $S(n_v, k) \leq B$. Modo heurístico: solo (i) y (ii).

_evaluar_k_particion(particion) ->None

Calcula EMD y guarda en `memoria_kparticiones`. Para $k = 2$: `bipartir().distribucion_marginal()` para $k > 2$: `np.mean(dists_por_grupo)`.

6.2. Métodos Principales de KQNodes

resolver() ->Solution

Prepara el oráculo Zeta, selecciona modo y retorna `Solution` con pérdida y partición formateada.

_exhaustive_k_partition(vertices, k) ->list[list]

Itera sobre el generador `_generar_particiones()` que produce las $S(n, k)$ particiones mediante backtracking recursivo.

_greedy_k_partition(vertices, k) ->list[list]

Ejecuta $k - 1$ iteraciones del MAO base, aislando el mejor corte en cada paso.

_compute_partition_loss(particion) ->float

$\delta_k(\Pi) = \text{EMD}(\bar{p}^\Pi, p_{\text{original}})$ donde $\bar{p}^\Pi = \frac{1}{k} \sum_i p(S_i)$.

_extraer_indices(grupo) ->(NDArray, NDArray)

Separa vértices de un grupo en índices futuros (EFECTO) y presentes (ACTUAL).

6.3. Dependencias Externas

Cuadro 4: Dependencias del proyecto K-QGMIP con su versión y uso.

Biblioteca	Versión	Uso principal
numpy	$\geq 2,0$	Operaciones matriciales, N-Cubos, EMD
plotly	$\geq 6,0$	Dashboard de resultados (HTML interactivo)
pandas	$\geq 2,0$	Lectura/escritura de resultados CSV
colorama	$\geq 0,4$	Colores en salida de terminal
pyinstrument	$\geq 5,0$	Profiling de rendimiento
pyttsx3	$\geq 2,99$	Síntesis de voz (anuncio de resultados)

6.4. Tests y Validación

TV-02 (KGeoMIP $k = 2 \equiv$ GeoMIP):

Verifica que `KGeoMIP(sub, k=2).ejecutar_k(2).perdida` $\approx$ `GeoMIP(sub).resolver().perdida` con tolerancia $< 10^{-6}$. Implementado en `src/shared/run/kgeomip.py`.

TV-03 (KQNodes $k = 2 \equiv$ QNodes):

KQNodes con $k = 2$ produce los mismos candidatos que el pre-pass de singletons de QNodes. Documentado en `kqnodes.py`.

Tests de integración:

`tests/test_qnodes_vs_phi.py` compara QNodes contra pyphi para un conjunto de casos fijos. Las diferencias esperadas están documentadas (pyphi usa EMD sobre repertorios completos, no marginales).

6.5. Manejo de Errores

- **Subsistema vacío:** Si `indices` o `dims` están vacíos, se retorna `Solution` con `perdida=0` inmediatamente.
- **Excepciones en `bipartir()`:** Capturadas silenciosamente en el bucle de evaluación; el grupo contribuye con distribución cero.
- **Modo heurístico:** Registrado en el logger con nivel `CRITIC` para trazabilidad y auditoría.

6.6. Uso de Inteligencia Artificial Generativa

Durante el desarrollo se utilizaron **Claude** (Anthropic) y **GitHub Copilot** en:

- **Diseño algorítmico:** Consultas sobre la formulación de la función de pérdida para $k > 2$ y cómo extender el oráculo Zeta al contexto de k -particiones.

- **Implementación:** Sugerencias para el generador recursivo `_generar_particiones()` y la optimización del promedio de distribuciones marginales con NumPy vectorizado.
- **Debugging:** Identificación de errores de indexación en la representación de vértices (ACTUAL/EFEECTO).
- **Documentación:** Asistencia en la redacción de docstrings y de este manual técnico.

Reflexión: La IA aceleró la prototipación pero requirió supervisión constante para respetar las convenciones del proyecto (notación little-endian, estructura de N-Cubos). Toda decisión algorítmica relevante fue verificada contra la teoría IIT.

7. Resultados Experimentales

7.1. Entorno de Experimentación

Las pruebas se ejecutaron con el orquestador `src/shared/run/benchmark.py` sobre redes sintéticas generadas en `src/shared/input/`. El hardware utilizado fue un equipo de escritorio con procesador Intel Core i7 y 16 GB de RAM, sistema operativo Windows 11. Cada instancia se ejecutó con semilla NumPy fija para reproducibilidad.

7.2. Conjuntos de Datos

Cuadro 5: Redes de prueba utilizadas en el benchmark. n : número de nodos.

Red	Tipo	n	Estados (2^n)
N5A	Pequeña A	5	32
N5B	Pequeña B	5	32
N10A	Mediana A	10	1,024
N10B	Mediana B	10	1,024
N15A	Grande A	15	32,768
N20A	Grande B	20	1,048,576
N21A	Grande C	21	2,097,152
N22A	Grande D	22	4,194,304
N23A	Grande E	23	8,388,608

7.3. Estrategias Evaluadas

Se ejecutaron las 10 estrategias sobre cada red: GeoMIP, KGeoMIP ($k = 2, 3, 4, 5$), QNodes, y KQNodes ($k = 2, 3, 4, 5$). Los resultados se guardan en `src/shared/output/{RED}/{ESTRATEGIA}` con columnas: `index`, `loss`, `time`, `partition`.

7.4. Validación de Consistencia (TV-02 / TV-03)

La Tabla ?? confirma que para $k = 2$, las extensiones son numéricamente equivalentes a las versiones base.

Cuadro 6: Validación TV-02/TV-03: diferencias de pérdida $|\delta_2^K - \delta_2^{\text{base}}|$.

Red	Par	Modo	Diferencia
N5B	KGeoMIP(2) vs GeoMIP	Exacto	$< 10^{-6}$
N5B	KQNodes(2) vs QNodes	Exacto	$< 10^{-6}$
N10A	KGeoMIP(2) vs GeoMIP	Heurístico	$< 10^{-4}$
N10A	KQNodes(2) vs QNodes	Greedy	$< 10^{-4}$
N15A	KGeoMIP(2) vs GeoMIP	Heurístico	$< 10^{-3}$
N15A	KQNodes(2) vs QNodes	Greedy	$< 10^{-3}$

7.5. Tiempos de Ejecución

Cuadro 7: Tiempo promedio de ejecución (segundos) por estrategia y red.

Estrategia	N5B	N10A	N10B	N15A	N20A	N22A
GeoMIP	< 0,01	0,05	0,06	0,31	2,1	9,4
KGeoMIP($k = 2$)	< 0,01	0,05	0,06	0,32	2,1	9,5
KGeoMIP($k = 3$)	0,02	0,12	0,13	0,85	5,3	22,1
KGeoMIP($k = 4$)	0,04	0,21	0,22	1,50	9,1	38,4
KGeoMIP($k = 5$)	0,07	0,38	0,40	2,70	16,2	67,9
QNodes	< 0,01	0,08	0,09	0,55	3,8	15,7
KQNodes($k = 2$)	< 0,01	0,08	0,09	0,56	3,8	15,7
KQNodes($k = 3$)	0,03	0,24	0,25	1,65	11,4	47,0
KQNodes($k = 4$)	0,06	0,47	0,49	3,30	22,7	93,8
KQNodes($k = 5$)	0,10	0,79	0,82	5,49	37,8	156,1

7.6. Comparación de Pérdidas (Loss)

Cuadro 8: Pérdida EMD promedio por estrategia y red.

Estrategia	N5B	N10A	N15A	N20A
GeoMIP	0,214	0,381	0,412	0,447
KGeoMIP($k = 3$)	0,118	0,220	0,251	0,281
KGeoMIP($k = 4$)	0,072	0,148	0,175	0,200
KGeoMIP($k = 5$)	0,043	0,103	0,131	0,159
QNodes	0,201	0,372	0,405	0,440
KQNodes($k = 3$)	0,110	0,211	0,244	0,275
KQNodes($k = 4$)	0,065	0,141	0,169	0,196
KQNodes($k = 5$)	0,038	0,097	0,126	0,154

7.6.1. Observación Principal

La pérdida EMD decrece consistentemente al aumentar k : la partición en más grupos aísla mejor cada variable, reduciendo la información destruida. Para $k = 5$ en redes grandes (N20A), KQNodes obtiene pérdidas $\sim 65\%$ menores que la bi-partición base.

7.7. Dashboard de Visualización

El archivo `src/shared/run/dashboard.py` genera un reporte HTML interactivo (`dashboard.html`) con tres secciones de gráficas Plotly:

1. **Tiempo por instancia:** líneas de tiempo de ejecución para cada estrategia sobre todos los casos del benchmark.

- 2. Correlación de pérdidas:** diagramas de dispersión para los 6 pares de estrategias (GeoMIP vs KGeoMIP(k), QNodes vs KQNodes(k), etc.).
- 3. Boxplots de tiempo:** distribución estadística del tiempo de ejecución por estrategia.

8. Limitaciones y Trabajo Futuro

8.1. Limitaciones Actuales

Modo heurístico no garantiza optimalidad.

Cuando $S(n, k) > 10,000$, ambas estrategias recorren solo un subconjunto de k -particiones candidatas. En redes con alta simetría, la heurística puede perder la partición óptima.

Escalabilidad de KQNodes greedy.

El modo greedy ejecuta $k - 1$ iteraciones completas del MAO. Para $k = 5$ y $n = 22$, el tiempo supera los 150 s, lo que puede resultar prohibitivo para exploraciones interactivas.

Función de pérdida promediada.

La extensión $\bar{p}^{\Pi} = \frac{1}{k} \sum_i p(S_i)$ asume que todos los grupos tienen igual peso. Una versión ponderada podría ser más fiel a la teoría IIT original [?].

Restricción a sistemas binarios.

La representación N-Cubo asume variables binarias. Sistemas multi-valorados (ternarios o continuos) requerirían una refactorización profunda de las estructuras de datos.

Definición de EMD.

La implementación utiliza la norma L_1 sobre distribuciones marginales unidimensionales, no la EMD bidimensional o Wasserstein-2. Esto difiere parcialmente de pyphi, que trabaja sobre repertorios completos.

Validación solo para $k = 2$.

TV-02 y TV-03 comparan las extensiones contra las versiones base únicamente en modo $k = 2$. No existe un baseline externo para $k \geq 3$.

Soporte de Windows únicamente verificado.

El módulo pytttsx3 para síntesis de voz puede tener comportamientos distintos en macOS/Linux. Las rutas de archivos asumen separadores de Windows en algunos puntos.

8.2. Trabajo Futuro

- 1. Paralelización del modo exhaustivo.** El generador `_generar_particiones()` es inherentemente serial. Se puede paralelizar con `multiprocessing.Pool` para explotar múltiples núcleos y escalar a redes de $n \leq 20$ en modo exacto.
- 2. Poda Branch-and-Bound en KGeoMIP.** Incorporar límites inferiores basados en la geometría del hipercubo para descartar ramas enteras sin evaluación completa.
- 3. Validación contra pyphi.** Ampliar `tests/test_qnodes_vs_phi.py` para cubrir casos $k \geq 3$ y documentar las diferencias esperadas entre las implementaciones.
- 4. Función de pérdida ponderada.** Implementar $\bar{p}^{\Pi} = \sum_i w_i \cdot p(S_i)$ con pesos proporcionales al tamaño del grupo ($w_i = |S_i|/|V|$) y evaluar su impacto en las métricas de Φ .

5. **Interfaz web.** Desarrollar una interfaz Dash/Streamlit para cargar TPM, seleccionar parámetros, ejecutar estrategias y visualizar resultados sin necesidad de línea de comandos.
6. **Soporte multi-plataforma.** Eliminar las dependencias de pyttssx3 del núcleo del benchmark y mover la síntesis de voz a un módulo opcional.
7. **Exportación a JSON/HDF5.** Agregar formatos de salida estructurados (JSON o HDF5) que preserven metadatos de la partición óptima y permitan análisis post-hoc sin re-ejecutar el benchmark.

8.3. Reflexión sobre el Proceso de Desarrollo

El proyecto K-QGMIP representa un primer paso hacia algoritmos de particionamiento eficientes para IIT con $k > 2$. La principal lección aprendida fue que la calidad de la función de pérdida (definición del promedio de distribuciones marginales) tiene un impacto mayor en los resultados que la sofisticación del algoritmo de búsqueda: una heurística sencilla con una función de pérdida precisa supera a una búsqueda exhaustiva con una pérdida aproximada.

A. Demostración: Consistencia para $k = 2$

Proposición A.1. Sea $\Pi = \{S_1, S_2\}$ una bi-partición de V . Entonces la pérdida $\delta_2(\Pi)$ calculada por KGeoMIP o KQNodes es idéntica a la pérdida calculada por GeoMIP o QNodes respectivamente.

Demostración. Para $k = 2$ el camino de código en KGeoMIP delega directamente a `GeoMIP.find_mip()` sin modificación (línea del `if k == 2` en `ejecutar_k()`). En KQNodes, el modo exhaustivo con $k = 2$ genera las mismas bi-particiones que el pre-pass de singletons del MAO base. La función `_compute_partition_loss` con dos grupos produce $\text{EMD}(\text{bipartir}(S_1, S_2).distribucion_marginal(), p_{\text{orig}})$, que es exactamente `funcion_submodular` de QNodes para los mismos índices. $\square$ $\square$

B. Tablas de Resultados Completas

B.1. Red N5B — Todas las Estrategias

Cuadro 9: Resultados completos para la red N5B.

Estrategia	Modo	Pérdida	Tiempo (s)	Partición
GeoMIP	—	0,2140	0,008	$\{A, B\} \{C, D, E\}$
KGeoMIP($k = 2$)	Exacto	0,2140	0,009	$\{A, B\} \{C, D, E\}$
KGeoMIP($k = 3$)	Exacto	0,1180	0,021	$\{A\} \{B, C\} \{D, E\}$
KGeoMIP($k = 4$)	Exacto	0,0720	0,041	$\{A\} \{B\} \{C\} \{D, E\}$
KGeoMIP($k = 5$)	Exacto	0,0430	0,068	$\{A\} \{B\} \{C\} \{D\} \{E\}$
QNodes	—	0,2010	0,007	$\{A, B\} \{C, D, E\}$
KQNodes($k = 2$)	Exacto	0,2010	0,008	$\{A, B\} \{C, D, E\}$
KQNodes($k = 3$)	Exacto	0,1100	0,029	$\{A\} \{B, C\} \{D, E\}$
KQNodes($k = 4$)	Exacto	0,0650	0,063	$\{A\} \{B\} \{C, D\} \{E\}$
KQNodes($k = 5$)	Exacto	0,0380	0,101	$\{A\} \{B\} \{C\} \{D\} \{E\}$

B.2. Tabla de Números de Stirling

Cuadro 10: Valores de $S(n, k)$. Los valores en **rojo** superan el umbral $B = 10,000$.

n	$k = 2$	$k = 3$	$k = 4$	$k = 5$	$k = n$
4	7	6	1	—	1
5	15	25	10	1	1
6	31	90	65	15	1
8	127	966	1,701	1,050	1
10	511	9,330	34,105	55,980	1
12	2,047	86,526	$\sim 10^6$	$> 10^6$	1
15	16,383	$\sim 2 \times 10^6$	$> 10^7$	$> 10^8$	1
20	524,287	$> 10^8$	$> 10^{11}$	$> 10^{14}$	1

C. Pseudocódigo Auxiliar: Generador de k-Particiones

Algorithm 8: `_generar_particiones`: backtracking recursivo para $S(n, k)$ particiones.

Entrada: Elementos E , número de grupos k , grupos actuales g

Salida: Generador de k -particiones

```

1 si  $|E| = 0$  y  $|g| = k$  entonces
2   yield  $g$ 
3   retornar
4 si  $|E| = 0$  o  $|g| > k$  entonces
5   retornar // poda
6 para  $i \leftarrow 0$  hasta  $\min(k, |g| + 1) - 1$  hacer
7   si  $i < |g|$  entonces
8      $g' \leftarrow g$  con  $E[0]$  añadido al grupo  $i$ 
9   sino
10     $g' \leftarrow g$  con nuevo grupo  $\{E[0]\}$ 
11   para cada  $\Pi \in \text{generar\_particiones}(E[1:], k, g')$  hacer
12     yield  $\Pi$ 

```

D. Glosario de Términos

EMD *Earth Mover's Distance*. Métrica de distancia entre distribuciones de probabilidad, equivalente a la norma L_1 entre sus marginales.

IIT *Teoría de la Información Integrada*. Marco matemático para cuantificar la conciencia mediante Φ .

MAO	<i>Maximum Adjacency Ordering</i> . Heurística voraz para minimizar funciones submodulares sobre grafos.
MIP	<i>Minimum Information Partition</i> . La partición del sistema que minimiza la pérdida de información al reconstruir la dinámica conjunta desde las partes.
N-Cubo	Representación del espacio de estados $\{0, 1\}^n$ como hipercubo n -dimensional. Cada nodo del sistema tiene un N-Cubo con su TPM.
Stirling	Número $S(n, k)$: cantidad de formas de particionar n elementos en exactamente k grupos no vacíos.
TPM	<i>Transition Probability Matrix</i> . Matriz de probabilidades de transición del sistema entre estados consecutivos.
Φ	Medida de información integrada. Un sistema es consciente en la medida en que $\Phi > 0$.
Zeta oracle	Estructura que precomputa la transformada Zeta sobre $\delta = H - p$ permitiendo consultas $\mathcal{O}(1)$ amortizado al oráculo de QNodes.

Referencias

- [1] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [2] Masafumi Oizumi, Larissa Albantakis, and Giulio Tononi. From the phenomenology to the mechanisms of consciousness: integrated information theory 3.0. *PLOS Computational Biology*, 10(5):e1003588, 2014.
- [3] Giulio Tononi, Melanie Boly, Marcello Massimini, and Christof Koch. Integrated information theory: from consciousness to its physical substrate. *Nature Reviews Neuroscience*, 17(7):450–461, 2016.